

Apuntes de Códigos Para La Página Web de Memot

Integrantes: Sofia Mojica, Gabriel Zacarias, Leonel De Armas, Mateo Roberts.

HTML:

¿Para qué usamos HTML?

HTML fue el lenguaje con el que armamos toda la estructura de la página. Gracias a él hicimos el Inicio, Productos, Mis Temas, Chat IA, Perfil, Login y Registro.

Con HTML agregamos botones, imágenes, formularios, títulos, textos, menús y todo lo que aparece en pantalla.

HTML no se encarga del diseño ni de la lógica. El diseño lo hace CSS y la lógica la hace JavaScript o PHP.

<!DOCTYPE html>

```
<!DOCTYPE html>
```

¿Para qué sirve?

Siempre va en la primera línea del archivo.

Le dice al navegador que el documento está escrito en HTML5.

Nosotros nunca lo modificamos.

Siempre es exactamente igual.

<html>

```
<html lang="es">
```

¿Para qué sirve?

Es la etiqueta donde empieza toda la página.

Todo el código HTML tiene que estar adentro.

Nosotros además usamos:

```
lang="es"
```

Eso solamente indica que la página está escrita en español.

<head>

```
<head>
```

```
</head>
```

¿Para qué sirve?

Dentro del head van todas las configuraciones de la página.

Acá normalmente pusimos:

- meta
- title
- link
- script

Todo eso ayuda a que la página funcione correctamente.

<meta charset="UTF-8">

```
<meta charset="UTF-8">
```

¿Para qué sirve?

Permite escribir normalmente:

- á
- é
- í
- ó
- ú
- ñ

Sin esta línea algunos caracteres pueden verse mal.

Nosotros la pusimos en todas las páginas.

<meta name="viewport">

```
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

¿Para qué sirve?

Hace que la página se adapte mejor a distintos tamaños de pantalla.

Si alguien entra desde un celular, la página se acomoda automáticamente.

<title>

```
<title>Mi Perfil</title>
```

¿Para qué sirve?

Es el nombre que aparece arriba del navegador.

Cada página tiene un título distinto.

Ejemplos:

```
<title>Inicio</title>
<title>Productos</title>
<title>Mis Temas</title>
```

<link>

```
<link rel="stylesheet" href="../CSS/Style.css">
```

¿Para qué sirve?

Conecta el HTML con el CSS.

Si sacáramos esta línea, la página seguiría funcionando, pero perdería todos los colores y el diseño.

rel

```
rel="stylesheet"
```

Indica que estamos conectando una hoja de estilos.

href

```
href="../../CSS/Style.css"
```

Le dice al navegador dónde está el archivo.

Rutas

Durante el proyecto usamos muchas rutas.

Por ejemplo:

```
../CSS/Style.css
```

Los dos puntos (..) significan subir una carpeta.

Después entra en la carpeta CSS.

Y finalmente abre Style.css.

Otro ejemplo:

```
../PHP/Login.php
```

Sube una carpeta y entra a PHP.

Si el archivo está en la misma carpeta simplemente escribimos:

```
Perfil.php
```

<body>

```
<body>
```

```
</body>
```

Todo lo que el usuario ve está dentro del body.

Por ejemplo:

- imágenes
- botones
- textos

- formularios
- gráficos

<header>

<header>

</header>

Representa la parte superior de la página.

En Memot lo usamos para hacer la barra de navegación.

<nav>

<nav>

</nav>

Sirve para crear el menú.

Dentro pusimos todos los botones para cambiar de página.

Por ejemplo:

Inicio

Productos

Mis Temas

Chat IA

Perfil

<main>

<main>

</main>

Representa el contenido principal.

Todo lo importante de la página lo pusimos acá.

Por ejemplo:

el formulario del perfil

el gráfico

la tarjeta principal

los controles del dispositivo

<div>

`<div>`

`</div>`

Es una caja.

Sirve para agrupar elementos.

Por sí sola no hace nada.

Normalmente la usamos para después darle estilos con CSS.

Ejemplo:

`<div class="glass-card">`

`...`

`</div>`

Toda la tarjeta rosa del proyecto es un div.

class

`class="glass-card"`

Sirve para ponerle un nombre a un grupo de elementos.

Después CSS busca esa clase y le aplica el diseño.

Ejemplo:

`.glass-card{`

`}`

Todos los elementos que tengan esa clase recibirán el mismo estilo.

Nosotros usamos muchísimas clases.

Ejemplos:

```
class="navbar"  
class="menu-links"  
class="glass-card"  
class="btn-tema"
```

id

```
id="graficoSueno"
```

El id sirve para identificar un único elemento.

La principal diferencia con class es que normalmente un id no se repite.

Nosotros lo usamos principalmente para JavaScript.

Por ejemplo:

```
<canvas id="graficoSueno"></canvas>
```

Después JavaScript hacía:

```
document.getElementById("graficoSueno")
```

y encontraba ese canvas para dibujar el gráfico.

<h1>

```
<h1>
```

```
Mi Perfil
```

```
</h1>
```

Es el título principal.

Normalmente usamos uno solo por página.

<h2>

`<h2>`

Registrar Sueño

`</h2>`

Se usa para subtítulos.

En Memot aparecen varios.

<p>

`<p>`

Bienvenida nuevamente.

`</p>`

Representa un párrafo.

Lo usamos para casi todos los textos.

`<strong>`

Características

`</strong>`

Hace que una parte del texto se vea resaltada.

Generalmente aparece en negrita.

<a>

`<a href="Perfil.php">`

Mi Perfil

`</a>`

Sirve para crear enlaces.

Cuando el usuario hace clic cambia de página.

href

Es la dirección a la que llevará el enlace.

Ejemplo:

```
href="Perfil.php"
```

o

```
href="../Index.php"
```

```
<ul>
```

```
</ul>
```

Representa una lista.

Nosotros la usamos en el menú superior.

```
<li>
```

```
</li>
```

Representa cada elemento de la lista.

Cada botón del menú está dentro de un `li`.

```

```

Sirve para mostrar imágenes.

src

Es la dirección donde está guardada la imagen.

alt

```
alt="Prototipo Memot"
```

Es un texto alternativo.

Si la imagen no carga aparecerá ese texto.

<form>

```
<form action="../../../PHP/guardarSueno.php" method="POST">
```

```
</form>
```

Sirve para enviar información al servidor.

Nosotros lo usamos para:

- Login
 - Registro
 - Guardar las horas de sueño
-

action

```
action="../../../PHP/guardarSueno.php"
```

Indica qué archivo PHP recibirá los datos.

method

```
method="POST"
```

Indica cómo se enviarán los datos.

Nosotros usamos POST porque la información no aparece en la dirección del navegador.

<label>

```
<label>
```

```
Fecha
```

```
</label>
```

Sirve para indicar qué información tiene que escribir el usuario.

<input>

`<input>`

Es donde el usuario escribe o selecciona información.

type="text"

Permite escribir texto.

type="password"

Oculto la contraseña.

type="email"

Permite escribir un correo electrónico.

type="date"

Abre un calendario para elegir una fecha.

Nosotros lo usamos para registrar la fecha del sueño.

type="time"

Permite elegir una hora.

Lo usamos para la hora de dormir y la hora de despertarse.

type="range"

Crea una barra deslizante.

La usamos para:

- volumen
 - intensidad de la luz
-

type="color"

Abre un selector de colores.

Lo usamos para elegir el color del NeoPixel.

name

`name="fecha"`

Es uno de los atributos más importantes.

PHP recibe los datos usando exactamente ese nombre.

Ejemplo:

HTML

`name="fecha"`

PHP

`$_POST["fecha"]`

Si los nombres fueran distintos, PHP no encontraría la información.

required

`required`

Hace que ese campo sea obligatorio.

Si el usuario intenta enviar el formulario vacío, el navegador no lo deja.

<select>

`<select>`

`</select>`

Crea una lista desplegable.

La usamos para elegir la calidad del sueño.

<option>

```
<option value="5">
```

Excelente

```
</option>
```

Cada opción tiene un valor.

Ese valor después llega a PHP.

<button>

```
<button>
```

Guardar

```
</button>
```

Crea un botón.

Si está dentro de un formulario, al presionarlo envía todos los datos.

También usamos botones normales para cambiar de tema y seleccionar distintas opciones.

<canvas>

```
<canvas id="graficoSueno"></canvas>
```

Es un espacio vacío donde JavaScript puede dibujar.

Nosotros lo usamos con Chart.js para mostrar el gráfico de horas dormidas.

<script>

```
<script src="../../JS/Perfil.js"></script>
```

Sirve para conectar un archivo JavaScript.

También lo usamos para cargar Chart.js desde internet.

CSS:

¿Para qué usamos CSS?

CSS fue el lenguaje que usamos para darle diseño a toda la página. Gracias a CSS pudimos elegir los colores, las fuentes, el tamaño de las letras, la posición de cada elemento y hacer que todo se vea más prolijo.

Si elimináramos el CSS, la página seguiría funcionando, pero se vería completamente desordenada y sin estilo.

Selectores

Los selectores sirven para decirle a CSS qué elemento queremos modificar.

Selector por etiqueta

```
body{  
  
}
```

Modifica todos los <body>.

Otro ejemplo:

```
button{  
  
}
```

Modifica todos los botones.

Selector por clase

```
.glass-card{  
}
```

Se usa cuando en HTML escribimos:

```
<div class="glass-card">
```

Todos los elementos que tengan esa clase recibirán ese diseño.

Fue el selector que más usamos.

Selector por id

```
#mensaje{  
  
}
```

Se usa cuando el HTML tiene

```
id="mensaje"
```

Nosotros casi no lo usamos porque preferimos trabajar con clases.

*

```
{  
  
}
```

El asterisco significa "todos los elementos".

Nosotros lo usamos para empezar todas las páginas.

Ejemplo:

```
{  
margin:0;  
padding:0;  
box-sizing:border-box;
```

```
font-family:'Comic Neue';  
}
```

Así evitamos que cada navegador agregue espacios diferentes.

margin

```
margin:20px;
```

Agrega espacio **por afuera** del elemento.

Es como dejar distancia entre un objeto y otro.

Ejemplo:

```
margin-top:30px;
```

Deja espacio solamente arriba.

También usamos:

```
margin-bottom
```

```
margin-left
```

```
margin-right
```

padding

```
padding:20px;
```

Agrega espacio **por adentro**.

Hace que el contenido no quede pegado al borde.

Lo usamos muchísimo en las tarjetas.

width

```
width:300px;
```


Define el ancho.

También usamos

`width:100%;`

Significa que ocupa todo el ancho disponible.

height

`height:200px;`

Define la altura.

En las imágenes normalmente dejamos

`height:auto;`

Así no se deformaban.

max-width

`max-width:600px;`

Significa que el elemento nunca podrá ser más ancho que ese valor.

Lo usamos para las tarjetas principales.

Así no se hacían gigantes en pantallas grandes.

min-height

`min-height:100vh;`

Fue una propiedad muy importante.

Significa que la página siempre tendrá como mínimo el alto de toda la pantalla.

Gracias a eso el fondo ocupaba toda la ventana.

color

`color:#2c3e50;`

Cambia el color del texto.

background

`background:white;`

Cambia el color de fondo.

También puede usarse con degradados.

background-image

`background-image:url("../IMAGENES/memo-fondo.jpeg");`

Coloca una imagen como fondo.

Nosotros usamos la misma imagen en casi todas las páginas.

background-size

`background-size:cover;`

Hace que la imagen ocupe toda la pantalla.

Aunque haya que recortar una parte.

background-repeat

`background-repeat:no-repeat;`

Evita que la imagen se repita muchas veces.

background-position

`background-position:center;`

Centra la imagen.

background-attachment

`background-attachment:fixed;`

Hace que el fondo quede quieto cuando bajamos la página.

Da una sensación más moderna.

display

Es una de las propiedades más importantes.

Nosotros casi siempre usamos

`display:flex;`

display:flex

Convierte un elemento en un contenedor flexible.

Gracias a Flexbox pudimos acomodar elementos mucho más fácil.

Por ejemplo:

la barra superior

las tarjetas

el perfil

los botones

justify-content

Se usa junto con Flexbox.

Controla la posición horizontal.

Ejemplo:

```
justify-content:center;
```

Centra todo.

justify-content:space-between;

Deja un elemento a cada lado.

Lo usamos en la barra superior.

Así el logo quedó de un lado y el menú del otro.

align-items

También se usa con Flexbox.

Controla la posición vertical.

Ejemplo:

```
align-items:center;
```

Centra todos los elementos verticalmente.

flex-direction

```
flex-direction:column;
```

Hace que los elementos se acomoden uno debajo del otro.

En lugar de quedar uno al lado del otro.

gap

`gap:15px;`

Agrega separación entre elementos de Flexbox o Grid.

Es mucho más cómodo que usar márgenes.

display:grid

También usamos Grid.

Ejemplo:

`display:grid;`

Sirve para organizar elementos como una tabla.

grid-template-columns

`grid-template-columns:repeat(2,1fr);`

Divide el espacio en dos columnas iguales.

Lo usamos para los botones de los temas.

border

`border:none;`

Quita el borde.

También puede ponerse un borde.

Ejemplo:

`border:1px solid black;`

border-radius

`border-radius:20px;`

Redondea las esquinas.

Lo usamos prácticamente en todos los botones y tarjetas.

border-bottom-left-radius

`border-bottom-left-radius:15px;`

Redondea solamente esa esquina.

Lo usamos en la barra superior.

box-shadow

`box-shadow:0 4px 20px rgba(0,0,0,.1);`

Agrega una sombra.

Hace que los elementos parezcan elevados.

Fue una propiedad muy usada en Memot.

opacity

`opacity:.9;`

Hace un elemento un poco transparente.

cursor

`cursor:pointer;`

Hace que aparezca la mano al pasar sobre un botón.

transition

`transition:.3s;`

Hace que los cambios sean suaves.

Por ejemplo cuando el botón cambia de color.

Sin transition el cambio sería instantáneo.

transform

`transform:scale(1.08);`

Aumenta el tamaño.

Lo usamos para que algunos botones crezcan un poco cuando pasamos el mouse.

:hover

`button:hover{`

`}`

Se activa cuando el mouse pasa sobre un elemento.

Lo usamos muchísimo.

Ejemplo:

Los botones cambiaban de color.

El icono del perfil aumentaba de tamaño.

Las tarjetas tenían pequeños efectos.

position

Sirve para controlar la posición.

position:fixed

`position:fixed;`

Hace que el elemento quede fijo.

Lo usamos en la barra superior.

Aunque bajemos la página, sigue arriba.

position:relative

Sirve como referencia para otros elementos.

Nosotros casi no lo usamos.

top

`top:0;`

Ubica el elemento arriba.

left

`left:0;`

Lo pega al lado izquierdo.

z-index

`z-index:1000;`

Controla qué elemento queda por encima de otro.

Lo usamos para que la barra superior siempre estuviera adelante.

overflow

`overflow:hidden;`

Oculto lo que sobresale.

Lo usamos principalmente en imágenes.

object-fit

`object-fit:contain;`

Hace que la imagen mantenga sus proporciones.

Así no se deformó el prototipo de Memot.

backdrop-filter

`backdrop-filter:blur(4px);`

Es una de las propiedades más lindas que usamos.

Hace el efecto de vidrio.

En lugar de verse un color sólido, desenfoca el fondo que está detrás.

Por eso nuestras tarjetas parecían de cristal.

-webkit-backdrop-filter

`-webkit-backdrop-filter: blur(4px);`

Hace exactamente lo mismo.

La agregamos para que también funcione en algunos navegadores como Safari.

list-style

`list-style: none;`

Quita los puntitos de las listas.

Lo usamos en el menú.

text-decoration

`text-decoration: none;`

Quita el subrayado de los enlaces.

text-transform

`text-transform: uppercase;`

Convierte todas las letras en mayúsculas.

Lo usamos para el logo.

letter-spacing

`letter-spacing: .5px;`

Separa un poco las letras.

Hace que el texto se vea más prolijo.

font-size

`font-size:1.2rem;`

Controla el tamaño de la letra.

font-weight

`font-weight:700;`

Hace que la letra sea más gruesa.

font-family

`font-family:'Comic Neue', cursive;`

Permite elegir la tipografía.

Nosotros usamos Comic Neue en todo el proyecto.

Variables CSS

Las usamos para no repetir colores.

Ejemplo:

```
:root{  
  
--texto-oscuro:#2c3e50;  
  
}
```

Después:

```
color:var(--texto-oscuro);
```

Si algún día queremos cambiar ese color solamente modificamos la variable y automáticamente cambia en toda la página.

JavaScript:

¿Para qué usamos JavaScript?

JavaScript fue el lenguaje que usamos para darle funcionamiento a la página. Gracias a él pudimos cambiar textos, crear el gráfico de horas de sueño, pedir información al servidor, mostrar mensajes automáticamente y hacer que algunos botones reaccionaran cuando el usuario los tocaba.

Mientras que HTML arma la estructura y CSS hace el diseño, JavaScript es el que hace que todo cobre vida.

<script>

Antes de usar JavaScript hay que conectar el archivo.

```
<script src="../../JS/Perfil.js"></script>
```

¿Para qué sirve?

Le dice al navegador que cargue el archivo JavaScript.

Si no estuviera esta línea, el navegador nunca ejecutaría el código.

const

```
const ctx = document.getElementById("graficoSueno");
```

¿Para qué sirve?

const crea una variable que no va a cambiar durante el programa.

Nosotros la usamos cuando sabíamos que ese valor siempre iba a ser el mismo.

Por ejemplo:

- el gráfico
- un botón

- un formulario
-

let

```
let mensaje = "";
```

¿Para qué sirve?

También crea una variable.

La diferencia es que su contenido puede cambiar.

Por ejemplo:

```
let mensaje = "";
```

```
mensaje = "Dormiste poco.";
```

```
mensaje = "Dormiste muy bien.";
```

document

```
document
```

¿Para qué sirve?

Representa toda la página HTML.

Gracias a document podemos buscar elementos y modificarlos.

document.getElementById()

```
document.getElementById("mensajeMemo")
```

¿Para qué sirve?

Busca un elemento usando su id.

En HTML:

```
<p id="mensajeMemo"></p>
```

JavaScript encuentra ese elemento y después puede cambiarlo.

Nosotros lo usamos muchísimo.

innerHTML

```
document.getElementById("mensajeMemo").innerHTML = mensaje;
```

¿Para qué sirve?

Cambia el contenido de un elemento.

Si antes decía

Hola

y hacemos

```
innerHTML="Bienvenida"
```

automáticamente pasa a decir

Bienvenida

En Memot lo usamos para mostrar el promedio y el mensaje automático.

value

```
input.value
```

¿Para qué sirve?

Obtiene el valor que escribió el usuario.

Por ejemplo:

Si alguien mueve un slider hasta 70,

slider.value

vale 70.

fetch()

fetch("../PHP/obtenerGrafico.php")

¿Para qué sirve?

Es uno de los comandos más importantes que usamos.

Sirve para pedir información a un archivo PHP sin tener que recargar la página.

En nuestro proyecto, cuando abrimos el Perfil, JavaScript usa fetch para pedirle a PHP todas las horas de sueño guardadas.

then()

fetch(...)

.then(...)

¿Para qué sirve?

fetch() tarda un poquito en recibir la respuesta.

.then() espera hasta que llegue.

Cuando finalmente llegan los datos, recién ahí ejecuta el código.

Nosotros usamos dos.

.then(res => res.json())

.then(datos => {

})

=>

res =>

datos =>

¿Para qué sirve?

Es una función flecha.

Es una forma más corta de escribir funciones.

En lugar de escribir

```
function(res){  
}
```

escribimos

```
res => {  
}
```

Hace exactamente lo mismo.

response

res

¿Qué representa?

Representa la respuesta que devuelve PHP.

Nosotros normalmente le pusimos el nombre "res", pero podría llamarse de cualquier manera.

json()

res.json()

¿Para qué sirve?

Convierte la respuesta de PHP en datos que JavaScript pueda entender.

PHP envía un JSON.

JavaScript lo transforma en un objeto.

datos

```
.then(datos=>{  
  
})
```

¿Qué representa?

Es la información que vino desde PHP.

Por ejemplo:

`datos.fechas`

`datos.horas`

Objetos

Cuando escribimos

`datos.horas`

estamos accediendo a una propiedad del objeto.

Ejemplo:

`datos`

↓

`fechas`

`horas`

Arrays

Un array guarda muchos datos.

Ejemplo:

`[6,7,8,5,9]`

Nosotros usamos arrays para guardar todas las horas dormidas.

reduce()

`datos.horas.reduce((a,b)=>a+b,0)`

¿Para qué sirve?

Suma todos los números del array.

Ejemplo:

`[7,8,6]`

↓

`7+8+6`

↓

`21`

Nosotros lo usamos para calcular el promedio.

length

`datos.horas.length`

¿Para qué sirve?

Cuenta cuántos elementos tiene un array.

Ejemplo

[7,8,6]

length vale

3

Promedio

Nosotros calculamos el promedio así

```
const promedio =  
datos.horas.reduce((a,b)=>a+b,0)  
/ datos.horas.length;
```

Primero sumamos todas las horas.

Después dividimos por la cantidad de noches.

toFixed()

```
promedio.toFixed(1)
```

¿Para qué sirve?

Redondea un número.

Ejemplo

7.34654

↓

7.3

Nosotros mostramos solamente un decimal.

if

```
if(promedio<6){
```

```
}
```

¿Para qué sirve?

Permite tomar decisiones.

Si se cumple la condición ejecuta el código.

else if

```
else if(promedio<8)
```

¿Para qué sirve?

Permite seguir comparando.

Nosotros hicimos tres niveles.

Menos de 6 horas.

Entre 6 y 8.

Más de 8.

else

```
else{
```

```
}
```

¿Para qué sirve?

Se ejecuta cuando ninguna condición anterior se cumplió.

Menor que

<

Ejemplo

`promedio<6`

Mayor que

`>`

Ejemplo

`promedio>8`

Igual

`==`

Compara dos valores.

Estrictamente igual

`===`

Compara valor y tipo de dato.

Generalmente es el que más se recomienda usar.

new

`new Chart(...)`

¿Para qué sirve?

Crea un objeto nuevo.

Nosotros lo usamos para crear el gráfico.

Chart()

```
new Chart(ctx,{  
  ...  
})
```

¿Para qué sirve?

Pertenece a la librería Chart.js.

Gracias a ella no tuvimos que dibujar el gráfico manualmente.

Simplemente le pasamos los datos.

labels

```
labels:datos.fechas
```

¿Para qué sirve?

Son los nombres que aparecen abajo del gráfico.

En nuestro caso las fechas.

datasets

```
datasets:[  
  
  ]
```

¿Para qué sirve?

Contiene toda la información que se va a dibujar.

Por ejemplo:

las horas

el nombre de la serie

los colores

data

`data:datos.horas`

¿Para qué sirve?

Es la información que aparecerá en el gráfico.

Cada número representa las horas dormidas ese día.

responsive

`responsive:true`

¿Para qué sirve?

Hace que el gráfico cambie de tamaño automáticamente según la pantalla.

beginAtZero

`beginAtZero:true`

Hace que el eje Y empiece en cero.

max

`max:12`

Limita el eje vertical hasta 12 horas.

addEventListener()

```
boton.addEventListener("click",function(){  
  
});
```

¿Para qué sirve?

Espera que ocurra un evento.

Nosotros lo usamos para detectar cuando el usuario hace clic en un botón.

click

"click"

Representa un clic del mouse.

querySelector()

`document.querySelector(".btn-tema")`

¿Para qué sirve?

Busca el primer elemento que coincida con un selector.

Puede buscar:

clases

ids

etiquetas

querySelectorAll()

`document.querySelectorAll(".btn-tema")`

¿Para qué sirve?

Busca todos los elementos.

Nosotros lo usamos cuando había varios botones iguales.

forEach()

`botones.forEach(...)`

¿Para qué sirve?

Recorre todos los elementos de un array.

Por ejemplo:

Todos los botones de temas.

classList

`elemento.classList`

¿Para qué sirve?

Permite modificar las clases CSS desde JavaScript.

classList.add()

`classList.add("active")`

Agrega una clase.

classList.remove()

`classList.remove("active")`

La elimina.

classList.toggle()

`classList.toggle("active")`

Si la clase existe la elimina.

Si no existe la agrega.

style

`elemento.style.display="none";`

¿Para qué sirve?

Permite modificar el CSS desde JavaScript.

Nosotros lo usamos para mostrar u ocultar elementos.

display

`style.display="block"`

Muestra un elemento.

`style.display="none"`

Lo oculta.

PHP:

¿Para qué usamos PHP?

PHP fue el lenguaje que usamos para trabajar del lado del servidor. Gracias a PHP pudimos:

- registrar usuarios
- iniciar sesión
- mantener la sesión abierta
- guardar las horas de sueño
- leer datos de la base de datos
- enviar información a JavaScript

Mientras que JavaScript trabaja en el navegador, PHP trabaja en el servidor.

¿Para qué sirve?

Indica dónde empieza y termina el código PHP.

Todo lo que esté entre esas etiquetas será ejecutado por el servidor.

session_start()

Inicia una sesión.

La sesión permite que la página recuerde quién es el usuario aunque cambie de página.

Nosotros la usamos en:

- Perfil.php
- Mistemas.php
- Asistente.php
- guardarSueno.php

Ejemplo de Memot

Explicación

- `session_start()` inicia o recupera la sesión.
 - `$_SESSION["id_usuario"]` busca el id del usuario.
 - `isset()` comprueba si existe.
 - Si no existe, significa que el usuario no inició sesión.
 - `header()` lo envía al Login.
 - `exit()` detiene el archivo.
-

\$_SESSION

¿Para qué sirve?

Guarda información del usuario mientras la sesión está abierta.

Nosotros guardamos:

- id_usuario
- nombre
- email

Ejemplo

Después, en otra página:

y aparece el nombre del usuario.

`$_POST`

¿Para qué sirve?

Recibe los datos enviados desde un formulario.

Ejemplo completo

HTML

PHP

El `name="email"` del HTML debe ser igual al nombre usado en `$_POST`.

`isset()`

¿Para qué sirve?

Comprueba si una variable existe.

Nosotros la usamos para:

- ver si el usuario inició sesión
 - evitar errores
-

`empty()`

¿Para qué sirve?

Comprueba si una variable está vacía.

Sirve para validar formularios.

`header()`

¿Para qué sirve?

Redirecciona automáticamente a otra página.

Lo usamos después de:

- iniciar sesión

- registrarse
 - guardar el sueño
-

exit()

¿Para qué sirve?

Detiene completamente el archivo.

Siempre lo usamos después de `header()`.

echo

¿Para qué sirve?

Muestra información en la página.

Ejemplo de Memot

mysqli

¿Para qué sirve?

Conecta PHP con MySQL.

Sin esta conexión no podríamos guardar ni leer datos.

prepare()

¿Para qué sirve?

Prepara una consulta SQL segura.

Evita problemas de seguridad.

bind_param()

¿Para qué sirve?

Le pasa los datos reales a la consulta.

Las letras significan:

- **s** = texto (string)
- **i** = número entero (integer)
- **d** = decimal (double)

Como nombre, email y password son textos, usamos **sss**.

execute()

¿Para qué sirve?

Ejecuta la consulta.

Recién acá se guarda la información.

get_result()

¿Para qué sirve?

Obtiene el resultado de una consulta SELECT.

fetch_assoc()

¿Para qué sirve?

Toma una fila de la base de datos y la convierte en un arreglo asociativo.

Ejemplo

Si la tabla tiene:

id	nombre
----	--------

1	Sofía
---	-------

Después de `fetch_assoc()` podemos hacer:

y obtener:

while

¿Para qué sirve?

Recorre todas las filas de una consulta.

Lo usamos para leer todos los registros de sueño.

json_encode()

¿Para qué sirve?

Convierte datos de PHP a JSON.

Este comando fue clave en Memot.

PHP preparaba:

y después:

JavaScript recibía esa información con `fetch()` y armaba el gráfico.

include y require

¿Para qué sirven?

Permiten usar código de otro archivo.

Sirven para no repetir la conexión a la base de datos.

password_hash()

¿Para qué sirve?

Convierte la contraseña en un texto seguro antes de guardarla.

Así la contraseña real no queda visible en la base de datos.

password_verify()

¿Para qué sirve?

Comprueba si la contraseña escrita por el usuario coincide con la guardada.

Flujo completo de Memot

1. Registro

- HTML muestra el formulario.
- PHP recibe los datos con `$_POST`.
- `password_hash()` protege la contraseña.
- `prepare()`, `bind_param()` y `execute()` guardan el usuario.

2. Login

- PHP busca el usuario.
- `password_verify()` comprueba la contraseña.
- `$_SESSION` guarda la información del usuario.
- `header()` lo envía al Perfil.

3. Perfil

- `session_start()` recupera la sesión.
- `echo` muestra el nombre.
- El formulario envía los datos del sueño.

4. Guardar sueño

- `$_POST` recibe fecha y horas.
- MySQL guarda el registro.
- `header("Location: Perfil.php")` vuelve al perfil.

5. Gráfico

- JavaScript hace `fetch()`.
- PHP lee la base de datos.
- `json_encode()` envía los datos.

- JavaScript recibe el JSON.
- Chart.js dibuja el gráfico.

- `session_start()` = iniciar sesión.
- `$_SESSION` = recordar al usuario.
- `$_POST` = recibir datos del formulario.
- `header()` = redireccionar.
- `prepare()` = consulta segura.
- `bind_param()` = pasar los datos.
- `execute()` = ejecutar.
- `fetch_assoc()` = leer una fila.
- `json_encode()` = enviar datos a JavaScript.
- `password_hash()` = guardar contraseñas de forma segura.
- `password_verify()` = comprobar contraseñas.